

QUESTION NO: 25

PROBLEM UNDERSTANDING

- The system must continuously monitor sensor inputs (motion, smoke, door, etc.) at fixed intervals using a periodic timer.
- When a monitored value crosses a defined threshold, the system must raise an alarm condition and hand it off to a dedicated alarm-handling task.
- Two RTOS primitives are combined for this handoff: an event flag notifies the handler that "something happened", while a message queue carries the actual alarm data (sensor id, value, timestamp).
- The alarm handler task must stay blocked (consuming no CPU) until the event flag is signalled, then safely drain the queue and take action.

RTOS MECHANISMS USED

- Software Timer (periodic, period = 200 ms) to trigger sensor sampling
- Event Flag (ALARM_EVENT) for lightweight signalling
- Message Queue (AlarmQueue) for passing structured alarm data
- Two independent tasks: SensorMonitorTask (producer) and AlarmHandlerTask (consumer)

STRUCTURED OVERVIEW

GLOBAL DEFINITIONS	DATA STRUCTURE
• DEFINE THRESHOLD • DEFINE QUEUE_SIZE = 10 • DEFINE ALARM_EVENT (event flag bit) • DEFINE TIMER_PERIOD = 200 (ms)	• AlarmMessage { • sensor_id : INTEGER • value : FLOAT • timestamp : TIME • }

PSEUDOCODE

TIMER: PeriodicSampleTimer

```
BEGIN
  SET PERIOD = 200 ms
  SET MODE = REPEATING
  ON EXPIRY -> SIGNAL SensorMonitorTask
END
```

TASK: SensorMonitorTask

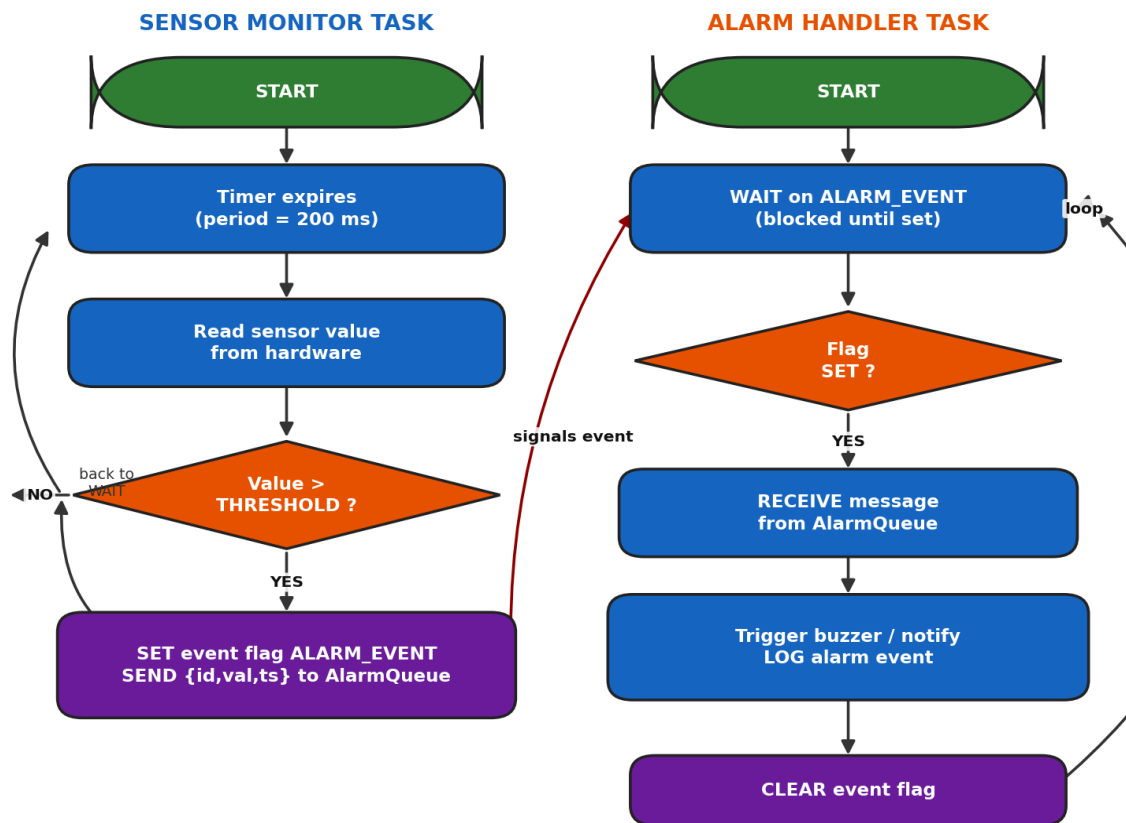
```
BEGIN
  DECLARE AlarmMessage msg
  WHILE TRUE
    WAIT UNTIL TimerSignal RECEIVED
    msg.value = READ SensorHardware()
    msg.sensor_id = SENSOR_ID
    msg.timestamp = GET_CURRENT_TIME()
    IF msg.value > THRESHOLD THEN
      SET EVENT FLAG ALARM_EVENT
      SEND msg TO AlarmQueue // non-blocking
    ENDIF
```

```
ENDWHILE  
END
```

TASK: AlarmHandlerTask

```
BEGIN  
DECLARE AlarmMessage received  
WHILE TRUE  
  WAIT ON EVENT FLAG ALARM_EVENT // BLOCK until set  
  RECEIVE received FROM AlarmQueue  
  ACTIVATE buzzer / notification  
  LOG received TO alarm_log  
  CLEAR EVENT FLAG ALARM_EVENT  
ENDWHILE  
END
```

LOGICAL FLOW DIAGRAM



QUEUE / RESOURCE OPERATIONS SUMMARY

Operation	Performed By	Purpose
SET(ALARM_EVENT)	SensorMonitorTask	Signal that an alarm condition occurred
SEND(AlarmQueue)	SensorMonitorTask	Deliver full alarm details to the handler
WAIT(ALARM_EVENT)	AlarmHandlerTask	Block until an alarm is raised (0% CPU while idle)
RECEIVE(AlarmQueue)	AlarmHandlerTask	Retrieve alarm details for processing
CLEAR(ALARM_EVENT)	AlarmHandlerTask	Reset the flag, ready for the next alarm

EXPLANATION

The SensorMonitorTask wakes up every 200 ms when the periodic timer expires, reads the sensor, and compares it against a threshold. If the value is abnormal, the task performs two actions together: it sets the ALARM_EVENT flag (a fast, low-overhead signal) and pushes the full alarm details onto the AlarmQueue. The AlarmHandlerTask stays blocked on the event flag so it uses no CPU time while idle, wakes instantly when the flag is set, and pulls the structured message from the queue to know exactly which sensor triggered the alarm and when. This combination gives the handler both a quick "wake-up call" (event flag) and the full context (queue) it needs to respond correctly.

KEY POINTS

- Event flags are used for signalling occurrence, not for transferring data.
- Queues carry the actual payload (sensor id, value, timestamp).
- The handler task performs a true RTOS blocking wait - it uses no CPU while idle.
- The timer decouples the sampling rate from task execution.

ASSUMPTIONS

- The sensor read operation is non-blocking and fast.
- The queue is large enough to hold at least one pending alarm; overflow is dropped/logged.
- Only one alarm condition type is modeled for simplicity; multiple sensors can reuse the same pattern.

REAL-LIFE EXAMPLE

A home security panel (e.g., a Honeywell or Bosch intrusion panel) samples door/window magnetic contact sensors and PIR motion sensors every 200 ms using a hardware timer interrupt. When a sensor trips, the firmware sets an event flag and queues the sensor ID and timestamp; a dedicated alarm-handling task, running at higher priority, wakes instantly, sounds the siren, and forwards the event to a cloud monitoring service - mirroring exactly this timer + queue + event-flag pattern.

APPLICATION

This pattern is the backbone of embedded alarm and monitoring systems - fire detection panels, industrial safety interlocks, tyre-pressure monitoring systems, and hospital patient-monitoring alarms - anywhere a system must react within milliseconds to a rare but critical event while spending most of its time idle to conserve power and CPU cycles.

QUESTION NO: 26

PROBLEM UNDERSTANDING

- Data must be sampled from an ADC at a precise, high-frequency rate (every 1 ms) - too fast and too timing-critical for a normal task, so it must happen inside an Interrupt Service Routine (ISR) triggered by a hardware timer.
- The ISR itself must do minimal work, per RTOS best practice - just capture the raw sample and hand it off, never perform heavy processing inside the interrupt.
- A separate task performs the actual filtering/processing at task-level priority, running only when new data is available.
- Synchronization between the ISR (producer) and the processing task (consumer) is done with a binary semaphore, while the data itself is transferred using a queue.

RTOS MECHANISMS USED

- Hardware Timer generating a periodic interrupt (1 ms)
- ISR (ADC_ISR) executing on every timer interrupt
- Binary Semaphore (DataReadySem) for ISR-to-task signalling
- Queues: DataQueue (raw samples) and LoggingQueue (processed batches)

STRUCTURED OVERVIEW

GLOBAL DEFINITIONS

- `DEFINE SAMPLE_PERIOD = 1 (ms)`
- `DEFINE QUEUE_SIZE = 50`
- `DEFINE BATCH_SIZE = 20`

DATA STRUCTURE

- Sample {
 - `raw_value : INTEGER`
 - `timestamp : TIME`
- }

PSEUDOCODE

TIMER: SamplingTimer

```
BEGIN
  SET PERIOD = SAMPLE_PERIOD
  SET MODE = REPEATING
  ON EXPIRY -> TRIGGER HARDWARE INTERRUPT -> ADC_ISR
END
```

ISR: ADC_ISR

```
BEGIN
  DECLARE Sample s
  s.raw_value = READ ADC_DATA_REGISTER
  s.timestamp = GET_TICK_COUNT()
  SEND s TO DataQueue      // ISR-safe, non-blocking
  SIGNAL DataReadySem      // give from ISR
  CLEAR INTERRUPT FLAG
  RETURN FROM INTERRUPT
END
```

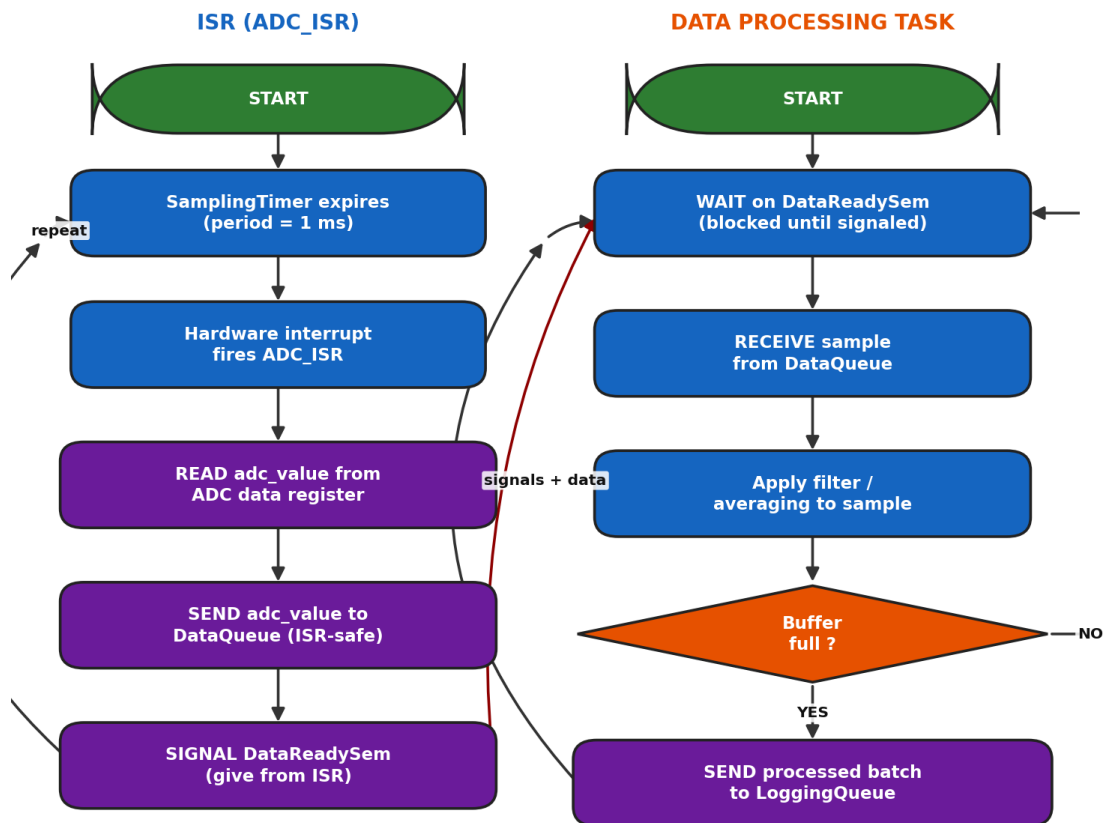
TASK: DataProcessingTask

```

BEGIN
DECLARE Sample s
DECLARE buffer[BATCH_SIZE]
DECLARE count = 0
WHILE TRUE
    WAIT ON DataReadySem    // BLOCK until signalled
    RECEIVE s FROM DataQueue
    filtered = APPLY_MOVING_AVERAGE(s.raw_value)
    buffer[count] = filtered
    count = count + 1
    IF count >= BATCH_SIZE THEN
        SEND buffer TO LoggingQueue
        count = 0
    ENDIF
ENDWHILE
END

```

LOGICAL FLOW DIAGRAM



QUEUE / RESOURCE OPERATIONS SUMMARY

Operation	Performed By	Purpose
SEND(DataQueue)	ADC_ISR	Hand the raw sample to the processing task
SIGNAL(DataReadySem)	ADC_ISR	Wake the processing task from ISR context
WAIT(DataReadySem)	DataProcessingTask	Block until a new sample is ready
RECEIVE(DataQueue)	DataProcessingTask	Retrieve the raw sample for filtering

Operation	Performed By	Purpose
SEND(LoggingQueue)	DataProcessingTask	Forward a full batch for storage/display

EXPLANATION

Every millisecond, the SamplingTimer fires a hardware interrupt that runs ADC_ISR. Because ISR execution time is extremely time-critical, it does the absolute minimum: read the ADC register, push the raw value onto DataQueue, and give the DataReadySem semaphore before returning. All the real work - filtering, averaging, batching - happens in DataProcessingTask, a normal task that spends most of its time blocked on the semaphore, so it consumes zero CPU when no data is pending. This ISR/semaphore/queue split is the standard RTOS pattern for high-rate sampling: keep interrupts short and defer processing to task context.

KEY POINTS

- ISRs must never call blocking APIs - only ISR-safe (non-blocking) queue/semaphore calls are used inside ADC_ISR.
- The semaphore is a pure synchronization signal; the queue carries the actual data.
- Deferring work from ISR to task context is called "interrupt bottom-half" processing.
- Batching before sending to LoggingQueue reduces the frequency of downstream I/O operations.

ASSUMPTIONS

- The ADC hardware supports a single-conversion read fast enough to complete within the ISR.
- DataQueue is large enough to absorb short bursts (queue size 50 vs. a 1 kHz sample rate).
- The processing task runs below interrupt priority (interrupts always preempt tasks) but above non-critical background tasks.

REAL-LIFE EXAMPLE

An ECG patient monitor samples the analog heart-signal at 1 kHz using a timer-triggered ADC interrupt. The ISR captures each raw sample and hands it to a processing task that applies a digital filter to remove noise and baseline wander before the waveform is drawn on screen and checked for arrhythmia - the same timer -> ISR -> semaphore -> queue -> task pipeline modeled here.

APPLICATION

This architecture underlies essentially all real-time data-acquisition (DAQ) systems: industrial vibration monitoring, audio sampling in digital signal processors, seismic sensor arrays, and biomedical instruments such as ECG/EEG monitors - anywhere raw signals must be captured at a fixed high rate without missing a sample, while heavier processing is safely deferred to task-level code.

QUESTION NO: 27

PROBLEM UNDERSTANDING

- Three tasks (TaskA, TaskB, TaskC) have equal priority, so priority comparison alone cannot decide which one runs next.
- Fairness must be achieved through Round-Robin (RR) time-slicing: each task gets a fixed CPU quantum (10 ms) before control is forcibly handed to the next task in circular order.
- The scheduler must use a periodic tick/timer interrupt to detect when a task's slice has expired and trigger a context switch.
- A task may also voluntarily give up the CPU early by yielding or blocking on I/O before its slice ends.

RTOS MECHANISMS USED

- Circular Ready Queue containing TaskA, TaskB, TaskC
- System slice timer (fires every 10 ms) driving preemption
- Context switch triggered on either timer expiry OR task yield/block

STRUCTURED OVERVIEW

GLOBAL DEFINITIONS

- DEFINE TIME_SLICE = 10 (ms)
- DEFINE NUM_TASKS = 3

DATA STRUCTURE

- TaskList[3] = {TaskA, TaskB, TaskC}
- index : INTEGER // pointer to running task

PSEUDOCODE

SCHEDULER: RoundRobinScheduler

```
DECLARE TaskList = [TaskA, TaskB, TaskC]
DECLARE index = 0
BEGIN
  WHILE TRUE
    current = TaskList[index]
    RESUME current          // STATE = RUNNING
    START sliceTimer FOR TIME_SLICE (10 ms)
    WAIT UNTIL (sliceTimer EXPIRES) OR (current YIELDS)
                    OR (current BLOCKS)
    IF current is still READY/RUNNING THEN
      SUSPEND current      // STATE = READY, requeue
    ENDIF
    index = (index + 1) MOD NUM_TASKS
  ENDWHILE
END
```

TASK: TaskA / TaskB / TaskC (identical pattern)

```
BEGIN
  WHILE TRUE
    PERFORM assigned work in small chunks
    IF work chunk complete before slice ends THEN
      YIELD() // voluntarily give up CPU early
    ENDIF
    // otherwise scheduler preempts automatically at 10 ms
```



```

ENDWHILE
END

```

TIMER ISR: sliceTimer_ISR

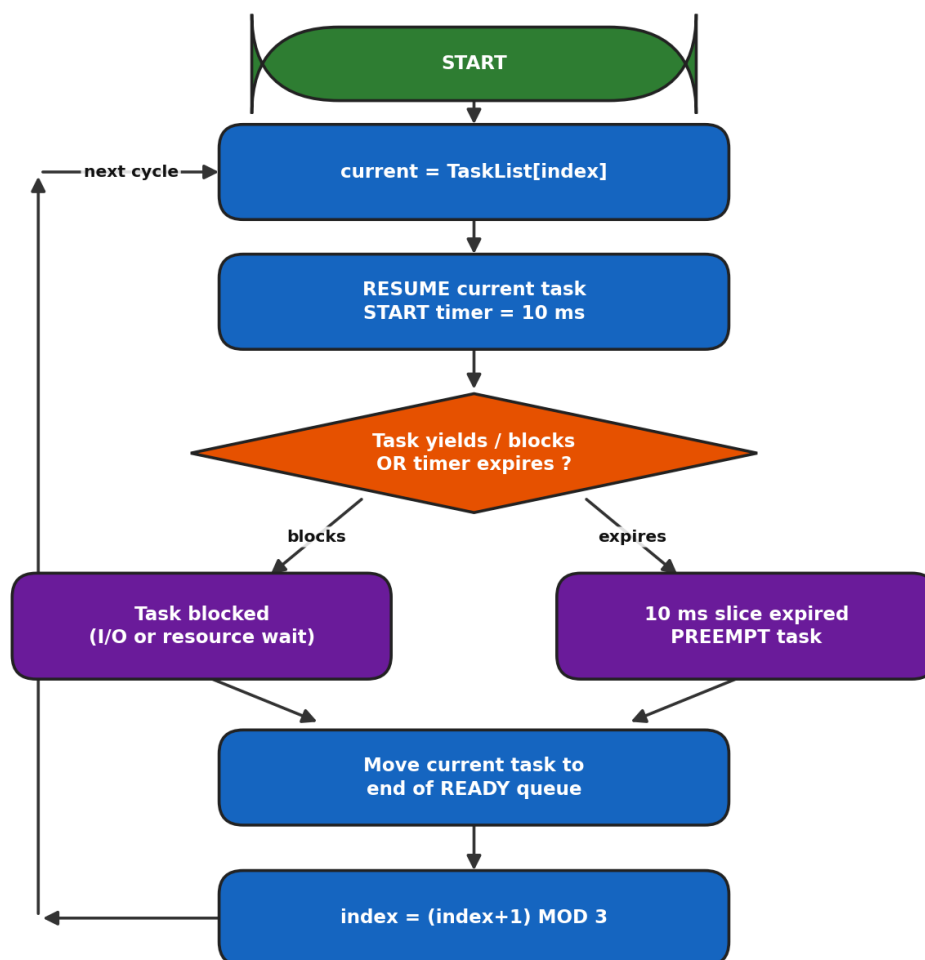
```

BEGIN
  SIGNAL scheduler "TIME_SLICE_EXPIRED"
  RESCHEDULE()
END

```

LOGICAL FLOW DIAGRAM

ROUND-ROBIN SCHEDULER (10 ms TIME SLICE)



Tasks cycle: Task A -> Task B -> Task C -> Task A ...

QUEUE / RESOURCE OPERATIONS SUMMARY

Operation	Performed By	Purpose
RESUME(current)	Scheduler	Dispatch the next task in circular order
START(sliceTimer)	Scheduler	Begin the 10 ms quantum for the running task
YIELD()	TaskA / TaskB / TaskC	Voluntarily release the CPU before slice ends
SUSPEND(current)	Scheduler	Move an expired task back to the READY queue

EXPLANATION

The scheduler keeps the three equal-priority tasks in a circular list and always executes them in the same fixed order: A, B, C, A, B, C ... Each time a task is dispatched, a 10 ms slice timer starts. If the timer expires before the task finishes or yields, the tick ISR forces a context switch - the task moves to READY and is placed at the back of the queue, and the next task begins running. A task can also give up its slice early by calling YIELD(), letting the next task start sooner. This guarantees that no task can monopolise the CPU and that all three receive an equal, predictable share of processing time.

KEY POINTS

- Round robin needs no priority comparison - only rotation order and slice expiry matter.
- The 10 ms slice must be tuned: too small increases context-switch overhead; too large hurts responsiveness.
- Preemption is timer-driven (tick ISR), independent of what the task is doing.
- A voluntary YIELD() lets a task be "polite" and let others run sooner.

ASSUMPTIONS

- All three tasks are CPU-bound and always have work available (no idle task modeled, for simplicity).
- Context-switch time is negligible compared to the 10 ms slice.
- No blocking I/O is involved in this pure CPU-scheduling illustration.

REAL-LIFE EXAMPLE

The POSIX real-time "SCHED_RR" scheduling class applies exactly this technique to give equal-priority threads - such as three independent sensor-polling threads in an IoT gateway - fair, bounded CPU turns so that no single polling thread can starve the others.

APPLICATION

Round-robin scheduling is widely used wherever several tasks of equal importance must share a processor fairly: multiplexing multiple network connections in a router, time-sharing the CPU among equal-priority background threads in an operating system, and cycling through multiple sensor-polling routines in a multi-sensor embedded controller.

QUESTION NO: 28

PROBLEM UNDERSTANDING

- The medical device runs multiple periodic tasks with different periods: HeartRateTask (20 ms), BloodPressureTask (50 ms), and AlarmCheckTask (100 ms).
- Rate-Monotonic Scheduling (RMS) is a static-priority scheme where priority is assigned inversely proportional to a task's period: the SHORTEST period gets the HIGHEST priority.
- The scheduler must be pre-emptive: whenever a higher-priority task becomes ready while a lower-priority task is executing, the lower-priority task is immediately pre-empted.
- Because this is a medical device, missed deadlines must be detected and logged, since patient safety depends on timely execution.

RTOS MECHANISMS USED

- Static priority assignment at task creation, following the RMS rule
- Pre-emptive, priority-based scheduler
- Periodic tasks using DELAY_UNTIL (absolute release time) to hold exact periods
- Deadline-miss detection and logging

STRUCTURED OVERVIEW

GLOBAL DEFINITIONS

- DEFINE HR_PERIOD = 20 (ms) PRIORITY = HIGH
- DEFINE BP_PERIOD = 50 (ms) PRIORITY = MEDIUM
- DEFINE ALARM_PERIOD = 100 (ms) PRIORITY = LOW

DATA STRUCTURE

- TaskControlBlock {
- period, priority,
- next_release_time, deadline
- }

PSEUDOCODE

INITIALIZATION: AssignPriorities (Rate-Monotonic Rule)

```
BEGIN
  HeartRateTask.priority = HIGH    // period 20ms (shortest)
  BloodPressureTask.priority = MEDIUM // period 50ms
  AlarmCheckTask.priority = LOW    // period 100ms (longest)
END
```

TASK: HeartRateTask (PRIORITY = HIGH, PERIOD = 20 ms)

```
BEGIN
  DECLARE next_release = GET_CURRENT_TIME()
  WHILE TRUE
    heart_rate = READ_HEART_SENSOR()
    IF heart_rate OUT OF SAFE RANGE THEN
      SEND ALERT TO AlarmQueue
    ENDIF
    next_release = next_release + 20
    DELAY_UNTIL(next_release)
  ENDWHILE
END
```

TASK: BloodPressureTask (PRIORITY = MEDIUM, PERIOD = 50 ms)

```
BEGIN
  DECLARE next_release = GET_CURRENT_TIME()
  WHILE TRUE
    bp = READ_BP_SENSOR()
    STORE bp IN PatientRecord
    next_release = next_release + 50
    DELAY_UNTIL(next_release)
  ENDWHILE
END
```

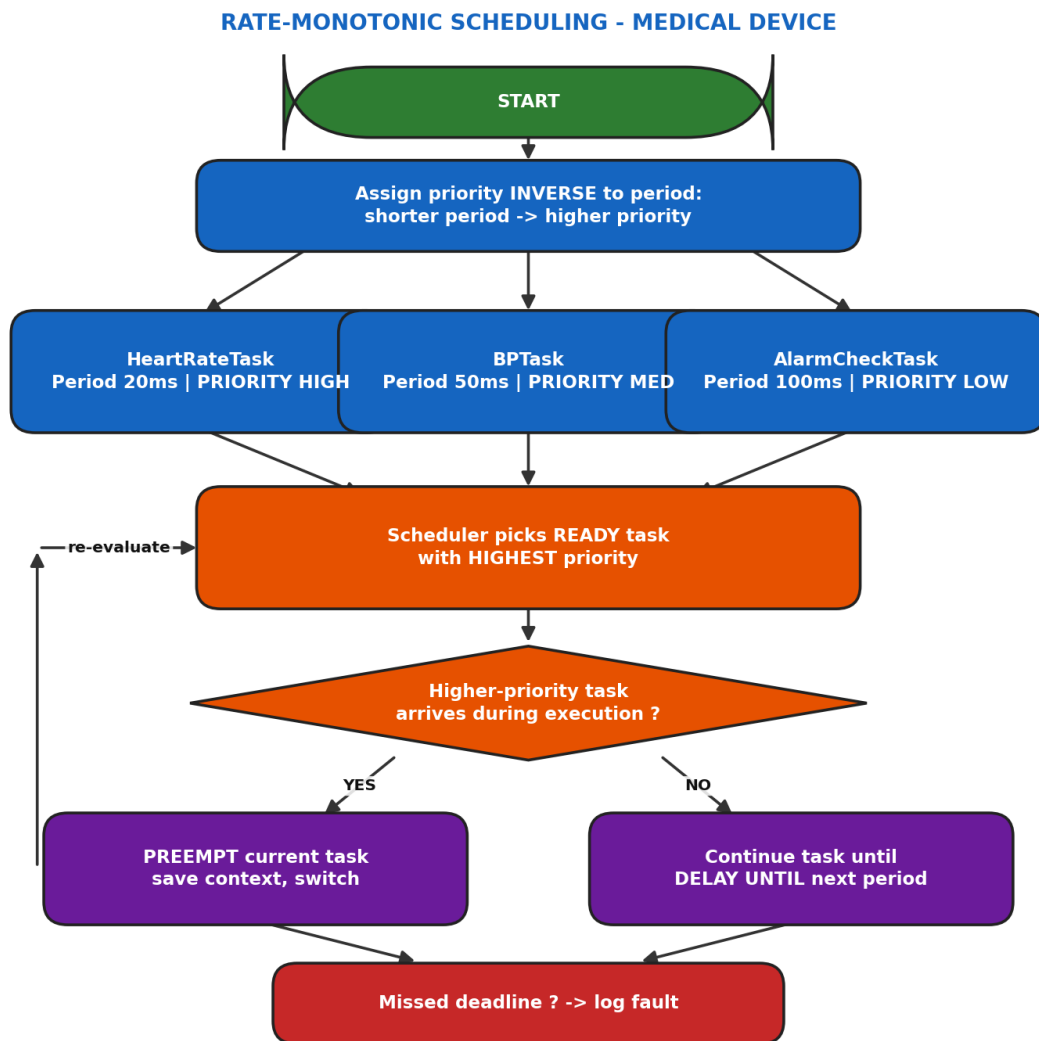
TASK: AlarmCheckTask (PRIORITY = LOW, PERIOD = 100 ms)

```
BEGIN
  DECLARE next_release = GET_CURRENT_TIME()
  WHILE TRUE
    IF AlarmQueue IS NOT EMPTY THEN
      RECEIVE alert FROM AlarmQueue
      TRIGGER audible / visual alarm
    ENDIF
    next_release = next_release + 100
    DELAY_UNTIL(next_release)
  ENDWHILE
END
```

SCHEDULER: PreemptivePriorityScheduler

```
BEGIN
  WHILE TRUE
    ready_task = HIGHEST PRIORITY task IN READY STATE
    RUN ready_task
    IF a higher-priority task's period elapses DURING execution THEN
      PREEMPT current task, SAVE context
      SWITCH to higher-priority task
    ENDIF
    IF task MISSES its deadline THEN
      LOG "deadline miss" WITH task name AND time
    ENDIF
  ENDWHILE
END
```

LOGICAL FLOW DIAGRAM



QUEUE / RESOURCE OPERATIONS SUMMARY

Operation	Performed By	Purpose
DELAY_UNTIL(t)	All periodic tasks	Hold an exact period, immune to preemption drift
SEND(AlarmQueue)	HeartRateTask	Report an out-of-range vital sign
RECEIVE(AlarmQueue)	AlarmCheckTask	Pick up a pending alert to annunciate
PREEMPT(task)	Scheduler	Suspend a lower-priority task for a higher-priority one

EXPLANATION

Rate-Monotonic Scheduling assigns fixed priorities purely from task periods: HeartRateTask (20 ms) is highest priority, BloodPressureTask (50 ms) is medium, and AlarmCheckTask (100 ms) is lowest. The scheduler always runs the highest-priority READY task; if HeartRateTask's period elapses while BloodPressureTask or AlarmCheckTask is executing, it immediately preempts them, because heart-rate monitoring cannot be delayed. Each task uses `DELAY_UNTIL` with an absolute release time (not a relative delay) so periods stay accurate even after preemption. Because this is safety-critical, the scheduler also checks whether any task has overrun its deadline and logs the event for the device's fault-monitoring subsystem.

KEY POINTS

- RMS priority rule: shorter period -> higher static priority, assigned once at task creation.
- RMS is provably optimal among static-priority algorithms for periodic tasks meeting the classic schedulability bound.
- `DELAY_UNTIL` (absolute delay) avoids period drift caused by preemption, unlike a plain `DELAY(period)`.
- Deadline-miss logging is essential in safety-critical / medical systems for post-incident analysis.

ASSUMPTIONS

- Tasks are independent, periodic, and pre-emptible, with deadlines equal to their periods.
- Each task's worst-case execution time is small enough relative to its period to satisfy the RMS schedulability test.
- Context-switch overhead is negligible.

REAL-LIFE EXAMPLE

Modern patient bedside monitors (e.g., GE / Philips ICU monitors) run heart-rate/ECG acquisition at the highest priority (sampled roughly every 20 ms), blood-pressure/SpO2 measurement at medium priority (less frequent), and alarm-annunciation logic at the lowest priority - precisely the rate-monotonic assignment shown here, so the most time-critical vital sign is never delayed by less urgent processing.

APPLICATION

Rate-Monotonic Scheduling is the standard technique for hard real-time embedded systems with multiple periodic tasks: avionics flight-control loops, automotive engine-control units (ECUs), and medical devices such as infusion pumps and patient monitors - wherever mathematically guaranteed, predictable timing is required for safety certification.

QUESTION NO: 29

PROBLEM UNDERSTANDING

- A home automation controller runs multiple tasks (LightControlTask, DoorSensorTask, ThermostatTask) managed by an RTOS scheduler.
- Each task, at any instant, exists in exactly one of four states: READY (able to run, waiting for the CPU), RUNNING (currently executing), BLOCKED (waiting on a resource/event/queue), or SUSPENDED (explicitly halted, excluded from scheduling until resumed).
- The pseudocode must model the transitions between these states as tasks interact with the scheduler and with shared resources such as an event queue.
- Understanding these transitions is central to reasoning about RTOS scheduling behaviour and debugging a stuck or unresponsive task.

RTOS MECHANISMS USED

- Task Control Block (TCB) with a 'state' field
- Pre-emptive priority scheduler moving tasks between READY and RUNNING
- Semaphore/queue WAIT calls moving RUNNING -> BLOCKED
- SuspendTask() / ResumeTask() API moving tasks to/from SUSPENDED

STRUCTURED OVERVIEW

GLOBAL DEFINITIONS	DATA STRUCTURE
• DEFINE ENUM TaskState = {READY, RUNNING, BLOCKED, SUSPENDED}	• TCB { • task_id : INTEGER • state : TaskState • priority : INTEGER • }

PSEUDOCODE

GLOBAL: Task Initialization

```
DEFINE ENUM TaskState {READY, RUNNING, BLOCKED, SUSPENDED}
```

```
TASK LightControlTask  STATE = READY
TASK DoorSensorTask    STATE = READY
TASK ThermostatTask    STATE = READY
```

SCHEDULER: StateDrivenScheduler

```
BEGIN
  WHILE TRUE
    selected = HIGHEST PRIORITY task WITH state == READY
    selected.state = RUNNING
    EXECUTE selected UNTIL ONE OF:
      (a) selected CALLS WAIT(resource)  -> state = BLOCKED
      (b) selected CALLS SuspendTask(self) -> state = SUSPENDED
      (c) slice expires / higher-pri ready -> state = READY
    MOVE selected TO matching queue (ready / blocked / suspended)
  ENDWHILE
END
```

TASK: DoorSensorTask

```
BEGIN
  WHILE TRUE
    IF DoorEventQueue IS EMPTY THEN
      WAIT ON DoorEventQueue  // RUNNING -> BLOCKED
    ENDIF
    RECEIVE event FROM DoorEventQueue  // BLOCKED -> READY -> RUNNING
    IF event == DOOR_OPENED THEN
      SEND command TO LightQueue
    ENDIF
  ENDWHILE
END
```

TASK: LightControlTask

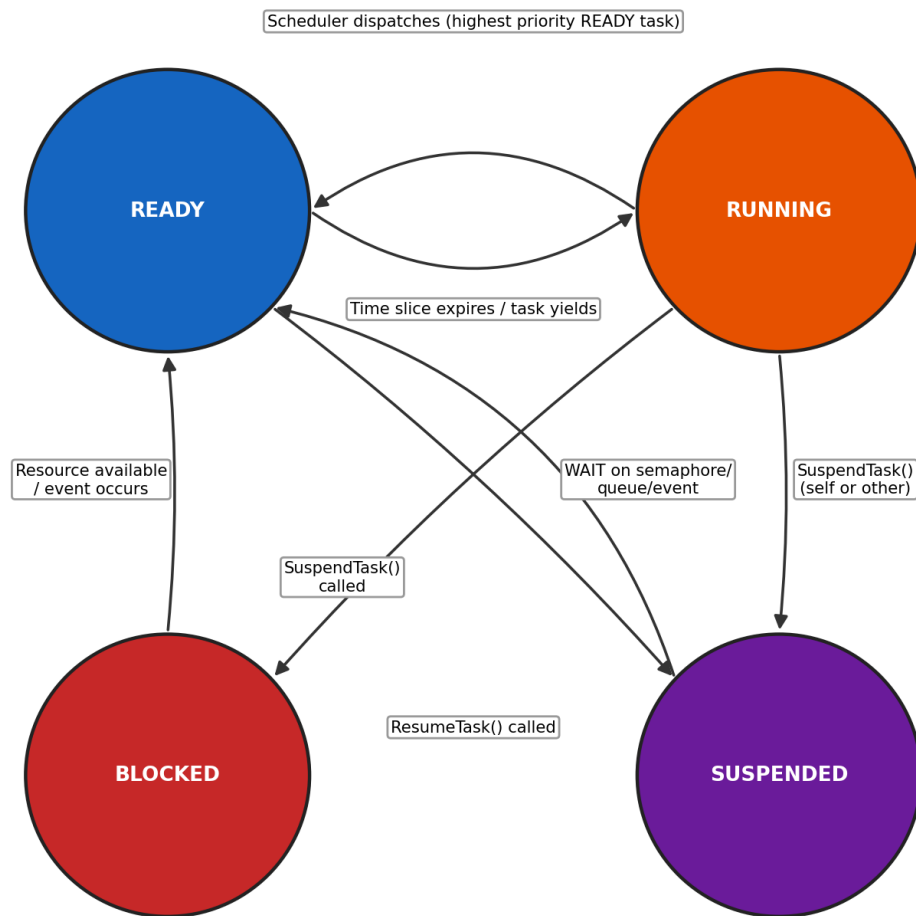
```
BEGIN
  WHILE TRUE
    RECEIVE command FROM LightQueue  // blocks if empty
    SWITCH light ON / OFF per command
  ENDWHILE
END
```

EVENT HANDLER: MaintenanceModeRequested

```
BEGIN
  SuspendTask(ThermostatTask)  // READY/RUNNING -> SUSPENDED
  ... perform maintenance ...
  ResumeTask(ThermostatTask)  // SUSPENDED -> READY
END
```


LOGICAL FLOW DIAGRAM

HOME AUTOMATION TASK STATE DIAGRAM



Tasks: LightControlTask, DoorSensorTask, ThermostatTask - each cycles through these states

QUEUE / RESOURCE OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT(DoorEventQueue)	DoorSensorTask	Move RUNNING -> BLOCKED when queue is empty
RECEIVE(DoorEventQueue)	DoorSensorTask	Move BLOCKED -> READY once an event arrives
SuspendTask(t)	Maintenance handler	Move a task to SUSPENDED regardless of activity
ResumeTask(t)	Maintenance handler	Move a SUSPENDED task back to READY

EXPLANATION

Every task begins life in READY. The scheduler promotes the highest-priority READY task to RUNNING. While running, DoorSensorTask may find its event queue empty and call WAIT, which moves it to BLOCKED - it consumes no CPU until a door event arrives, at which point it returns to READY and is eventually re-scheduled to RUNNING. Separately, a maintenance routine can explicitly call SuspendTask() on ThermostatTask, moving it to SUSPENDED regardless of what it was doing; a suspended task is completely excluded from scheduling until an explicit ResumeTask() call returns it to READY. This four-state model lets the RTOS distinguish "waiting for something to happen" (BLOCKED, expected to resume itself) from "deliberately paused" (SUSPENDED, requires an external resume).

KEY POINTS

- RUNNING <-> READY transitions are scheduler-driven (priority / time-slice based).
- RUNNING -> BLOCKED is self-initiated by the task waiting on a resource; BLOCKED -> READY is event-driven.
- SUSPENDED is unique: only an explicit SuspendTask()/ResumeTask() call can enter or leave it - the scheduler never does this automatically.
- On a single-core system, only one task can be RUNNING at any instant.

ASSUMPTIONS

- Single-core processor - only one task RUNNING at a time.
- SuspendTask() can be called on self or on another task by a higher-authority task/ISR.
- BLOCKED tasks always have a well-defined wake-up condition (no permanent blocking).

REAL-LIFE EXAMPLE

In a smart-home hub (e.g., a Google Home / SmartThings-style controller), the door-sensor task sits BLOCKED for most of the day, waiting on a Zigbee/Z-Wave event queue, and becomes READY the instant a door opens; meanwhile, if the homeowner puts the thermostat into "vacation mode" via the app, the firmware explicitly SUSPENDs the ThermostatTask until the mode is turned off - exactly the READY/RUNNING/BLOCKED/SUSPENDED behaviour modeled here.

APPLICATION

This four-state task model is fundamental to every general-purpose RTOS (FreeRTOS, VxWorks, Zephyr) and underlies debugging tools that answer "why is my task not running" - it applies directly to any embedded system with multiple cooperating tasks, from home-automation hubs to industrial controllers and wearable devices.